

State & Hooks

React State Management and Side Effects

■ LEARNING OUTCOME

By the end of this module you will use all essential React hooks and build custom hooks for reusable stateful logic

■ Skills You Will Gain

- Manage state with useState
- Handle side effects with useEffect
- Share global state with useContext
- Manage complex state with useReducer
- Use useRef for DOM access
- Optimise with useMemo and useCallback
- Build custom hooks

■ Hooks are what make React functional components powerful. `useState`, `useEffect`, `useContext`, `useReducer` -- these four hooks cover 90% of all React state needs.

SECTION 1

useState and useEffect

```
// useState: [value, setter] = useState(initial) function Counter() { const [count, setCount] =
useState(0); const [form, setForm] = useState({ name:'', email:'' }); const [items, setItems] =
useState([]); // Functional update -- use when new state depends on old const inc = () =>
setCount(prev => prev + 1); // Update object -- MUST spread entire object const updateField =
(field, value) => setForm(prev => ({ ...prev, [field]: value })); // Update array const addItem =
item => setItems(prev => [...prev, item]); const removeItem = id => setItems(prev => prev.filter(i
=> i.id !== id)); const updateItem = (id, ch) => setItems(prev => prev.map(i => i.id===id ?
{...i,...ch} : i)); return <button onClick={inc}>Count: {count}</button>; } // useEffect: run side
effects after render function UserProfile({ userId }) { const [user, setUser] = useState(null);
const [loading, setLoading] = useState(true); const [error, setError] = useState(null);
useEffect(() => { let cancelled = false; // prevent update after unmount setLoading(true);
fetch(`/api/users/${userId}`) .then(r => { if(!r.ok) throw new Error(r.statusText); return
r.json(); }) .then(d => { if(!cancelled) { setUser(d); setLoading(false); } }) .catch(e => {
if(!cancelled) { setError(e.message); setLoading(false); } }); return () => { cancelled = true; };
// CLEANUP }, [userId]); // run when userId changes // Dependency array: // [] -- run once on mount
// [x, y] -- run when x or y changes // (omitted) -- run after EVERY render }
```

SECTION 2

useContext, useReducer, useRef and Optimisation

```
// useContext -- shared state without prop drilling const ThemeCtx = createContext({ theme:'light',
toggle:()=>{} }); function ThemeProvider({ children }) { const [theme, setTheme] =
useState('light'); const toggle = useCallback(() => setTheme(t => t==='light'?'dark':'light'), []);
return <ThemeCtx.Provider value={{ theme, toggle }}>{children}</ThemeCtx.Provider>; } function
ThemeBtn() { const { theme, toggle } = useContext(ThemeCtx); return <button onClick={toggle}
className={`btn-${theme}`}>Toggle Theme</button>; } // useReducer -- complex state with named
actions const reducer = (state, action) => { switch(action.type) { case 'INC': return { ...state,
count: state.count+1 }; case 'DEC': return { ...state, count: state.count-1 }; case 'RESET': return
{ count:0, history:[] }; default: throw new Error('Unknown: '+action.type); } }; function Counter()
{ const [state, dispatch] = useReducer(reducer, {count:0, history:[]}); return <button onClick={()
=> dispatch({type:'INC'})}>Count: {state.count}</button>; } // useRef -- DOM access, persist value
without re-render function FocusInput() { const inputRef = useRef(null); return ( <> <input
ref={inputRef} type='text' /> <button onClick={() => inputRef.current?.focus()}>Focus</button> </>
); } // Custom hooks -- extract reusable logic function useFetch(url) { const [data, setData] =
useState(null); const [loading, setLoading] = useState(true); const [error, setError] =
useState(null); useEffect(() => { let cancelled = false; fetch(url).then(r=>r.json()).then(d=>{
if(!cancelled) { setData(d); setLoading(false); } }) .catch(e=>{ if(!cancelled) {
setError(e.message); setLoading(false); } }); return () => { cancelled = true; }; }, [url]); return
{ data, loading, error }; } // useMemo and useCallback for performance const sorted = useMemo(() =>
items.sort((a,b)=>a.price-b.price), [items]); const handle = useCallback((id) => doSomething(id),
[]); const MemoChild = React.memo(({ onClick }) => <button onClick={onClick}>Click</button>;
```

■ PRACTICE Q & A

Q1. When does a `useEffect` run?

A: After every render by default. With `[]`: once on mount. With `[dep1]`: when `dep1` changes. Cleanup runs before next effect and on unmount.

Q2. What is the difference between `useMemo` and `useCallback`?

A: `useMemo` memoizes a **VALUE**. `useCallback` memoizes a **FUNCTION REFERENCE**. Use `useMemo` for expensive calculations; use `useCallback` for stable function props to memoized children.

Q3. When to use `useReducer` instead of `useState`?

A: When: state has multiple sub-values that change together, next state depends on previous in complex ways, or state transitions have business logic.

Q4. What is the stale closure problem?

A: A closure captures values at creation. Inside useEffect, referencing state without including it in deps gives old values. Fix: add to dependency array or use functional updates.

Q5. What are the rules of hooks?

A: 1. Only call hooks at TOP LEVEL (not in loops or conditions). 2. Only call from React function components or custom hooks.

■ Hooks Cheat Sheet	
<code>const [val, setVal] = useState(init)</code>	Local state
<code>setVal(prev => prev + 1)</code>	Functional update
<code>useEffect(() => {}, [dep])</code>	Effect with dependencies
<code>return () => cleanup</code>	Effect cleanup function
<code>useContext(MyContext)</code>	Consume context
<code>useReducer(reducer, initial)</code>	Complex state
<code>useRef(null)</code>	DOM ref or mutable value
<code>ref.current.focus()</code>	DOM method via ref
<code>useMemo(() => value, [deps])</code>	Cache computed value
<code>useCallback(() => fn, [deps])</code>	Stable function ref
<code>React.memo(Component)</code>	Skip render if props unchanged
<code>function useCustom() {}</code>	Custom hook naming